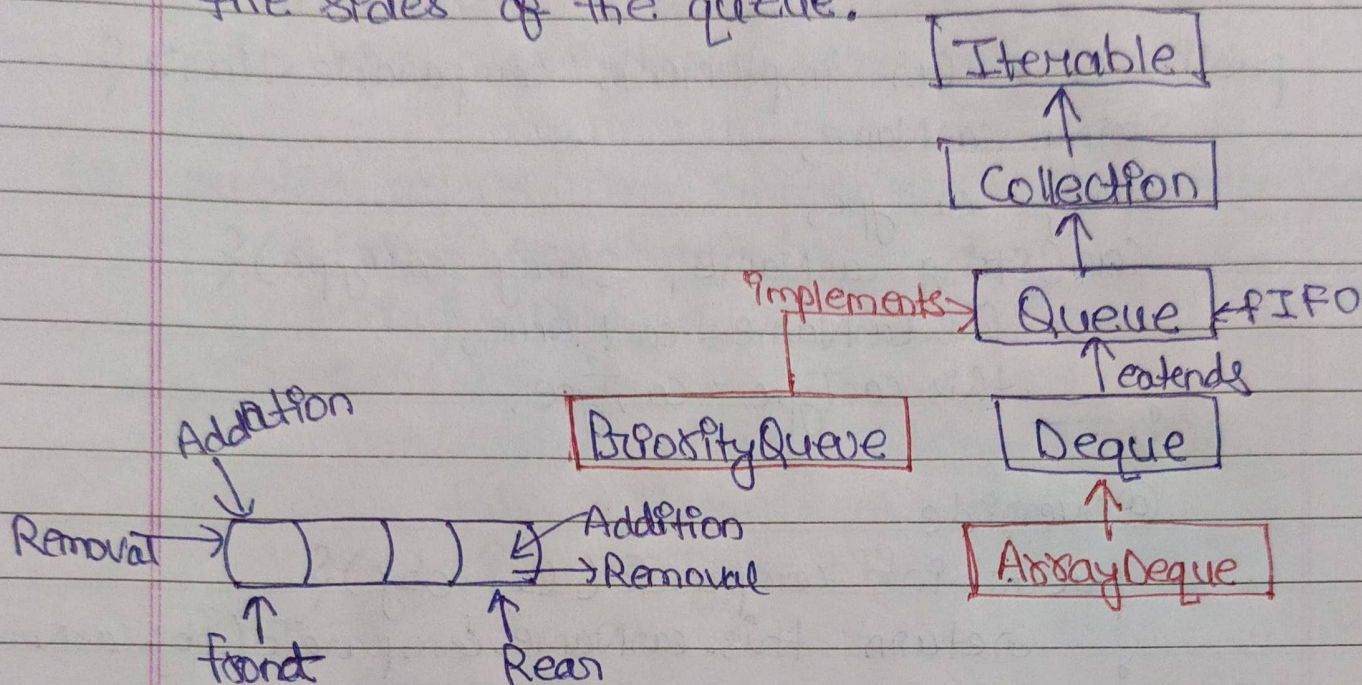


Deque

Page No. :

Date : / /

Deque: Stands for Double Ended Queue. Means addition and removal can be done from both the sides of the queue.



Methods available in Deque Interface:

→ It contains Methods available in Collection as well as Queue Interface + Method available in Deque Interface.

Queue Methods like:

add(), offer(), poll(), remove(), peek(), element().

	Insertion Opn	Removal Opn	Examine Operation
ThrowException	addFirst(e)	removeFirst()	getFirst()
DoNotThrowExcpn	offerFirst(e) return true/false	pollFirst() return null if deque empty	peekFirst() return null if deque empty
ThrowException	addLast(e)	removeLast()	getLast()
DoNotThrowExcpn	offerLast(e) return T/F	pollLast() return null if deque empty	peekLast() return null if deque empty

- Using these methods, we can even use Deque to implement STACK (LIFO) and Queue (FIFO) both.
- To use it as Stack, push() and pop() methods are available
- push() → Internally call addFirst()
- pop() → Internally call removeLast()

Queue methods

- add() → Internally calls addLast() method
- offer() → calls offerLast() method
- poll() → calls pollFirst() "
- remove() → calls removeFirst() method
- peek() → calls peekFirst() method
- element() → calls getFirst() method.

ArrayDeque:

→ ArrayDeque (concrete class), implements the methods which are available in Queue and Deque interface.

```
Ex: public class Main {
```

```
    psvm() {
```

```
        ArrayDeque<Integer> dqAsQueue = new ArrayDeque<>();
```

```
        // Insertion
```

```
        dqAsQueue.addLast(1);
```

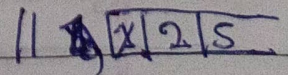
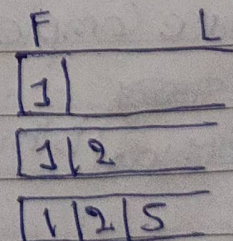
```
        dqAsQueue.addLast(2);
```

```
        // addLast(5);
```

```
        // Deletion
```

```
        int el = dqAsQueue.removeFirst();
```

```
        Sout(el); // 1
```



// LIFO

ArrayDeque<Integer> dqAsStack = new ArrayDeque<>(2);

dqAsStack.addFirst(1);

[1]

dqAsStack.addFirst(5);

[5|1]

——.addFirst(10);

[10|5|1]

// Deletion

int val = dqAsStack.removeFirst();

[10|5|1]
F L

cout(val); // 10

}

}

Time Complexity:

- Insertion : Amortized (Most of the time or Average), complexity is $O(1)$ except few cases like $O(N)$: when queue size threshold (default is 8) reached and try to insert an element at end or front, then its $O(N)$ bigger size

- Deletion : $O(1)$

- Search : $O(1)$ // peek or examine operation

Space Complexity : $O(n)$

Queue Impl through Collection Framework	Is thread safe	Maintaining Insertion order	Null elements allowed	Duplicate elements allowed
PriorityQueue	No	No	No	Yes
ArrayDeque	No	Yes	No	Yes

How to make Deque thread safe version

i) PriorityQueue

Ex: public class Main {

psum() {

PriorityQueue<Integer> pq =

new PriorityQueue<>();

↑ by default minheap

pq.add(5);

pq.add(2);

System.out.println(pq.peek()); // 2

}

}

ii) ConcurrentLinkedDeque

public class Main {

psum() {

ConcurrentLinkedDeque<Integer> dq =

new ConcurrentLinkedDeque<>();

dq.addFirst(2);

dq.addLast(1);

System.out.println(dq.removeLast()); // 1